

Modelo de datos y reglas de negocio

Ingeniería de Sistemas e Informática · Universidad Continental
Expediente organizado v1.0 | 22/09/2026

Modelo conceptual y relaciones

Entidades y cardinalidades

Entidad	Relación conceptual	Observación
Usuario	1 usuario tiene 0..1 perfil técnico y 0..N sesiones	Roles: cliente, técnico y administrador
Cliente	1 cliente registra 0..N solicitudes	Cada solicitud pertenece a un cliente
Técnico	1 técnico recibe 0..N ofertas	Perfil con especialidades y cobertura
Solicitud	1 solicitud genera 0..N ofertas	0..1 técnico asignado en el modelo actual
Oferta	Relaciona una solicitud con un técnico	Pareja única; plazo y estado propios
Evento	0..N eventos por solicitud	Actualmente JSON dentro de requests
Calificación	0..1 puntuación por solicitud finalizada	Actualmente atributo de requests, no tabla aparte

Conceptual frente a físico

El modelo conceptual explica el negocio. El físico observado usa users, api_tokens, technicians, requests y offers. Especialidades, zonas y eventos usan JSON. Normalizarlos en tablas auxiliares sería una evolución, no un hecho de esta versión.

Fuente de instalación

Usar migraciones de backend/api/database/migrations. database/init.sql es esquema de referencia: no ejecutarlo y después volver a crear las mismas tablas con migraciones sobre la misma base.

Diccionario resumido y reglas

Datos principales

Tabla	Campos relevantes	Control
users	id, name, email, password, role	Correo único; contraseña hash; rol permitido
api_tokens	user_id, token_hash, expires_at	Token hash único, vencimiento y revocación
technicians	user_id, zone, zones, specialties, fee, rating, reviews, years	Usuario único; verificado y disponible
requests	client_id, specialty, zone, address, description, status, technician_id	Propiedad, transiciones y asignación
requests	created_ms, pending_until, completed_at, rating_until, rating, events	Plazos y nota única
offers	request_id, technician_id, status, sent_at, expires_at, score	Unicidad request_id + technician_id

Ranking

Filtrar verificados, disponibles, especialidad y cobertura. Cercanía: 1 en zona principal, 0,5 en adicional; tarifa inversa normalizada entre elegibles; calificación / 5; experiencia limitada a 10 años / 10. Pesos 0,35/0,20/0,30/0,15. Redondeo documentado a seis decimales; desempate por calificación y luego identificador.

Reglas temporales

No aceptar oferta vencida; no volver a notificar al mismo técnico para esa solicitud; reservar al aceptar y liberar al cancelar antes de ejecución o finalizar; rechazar calificación tardía y duplicada. Fuente común: domain/rules.json y servicios del backend.

Integridad y evolución

Pruebas de base de datos

Verificar claves y referencias, unicidad de correo y oferta, rechazo de datos inválidos en API, persistencia de estados, atomicidad y concurrencia real. Probar migración en base vacía y sobre copia de la versión anterior, comparando conteos, relaciones y datos significativos.

Seguridad y privacidad

El cliente accede a sus solicitudes; el técnico a ofertas y atenciones autorizadas; administración supervisa según permisos. Evitar direcciones ajenas, contraseñas, tokens y datos reales en logs o capturas. Retención y eliminación requieren política aprobada, no se presume plazo legal.

Respaldo y recuperación

Antes de un cambio de esquema: identificar build, crear respaldo controlado y comprobar restauración en entorno aislado. Registrar RPO y RTO objetivo antes del ensayo. No ejecutar migraciones destructivas o rollback sobre datos reales como experimento.

Cambios propuestos

Evaluar índices según consultas y carga; separar eventos y especialidades cuando exista necesidad justificada; definir auditoría persistente de acciones administrativas. Cada cambio requiere decisión arquitectónica, migración versionada, caso de integridad y actualización del diccionario.

Fuentes

Base documental: fuentes/proyecto-final.pdf; fuentes/plan-de-pruebas-original.pdf; fuentes/ejemplos-practica.docx; fuentes/estado-del-arte-original.pdf. Código: rutas relativas a la raíz del repositorio. Fuentes conservadas con SHA-256 en fuentes/manifest.json. Esquema inspeccionado: database/init.sql.